

PHYS 134L Spring 2025 Lab 5

Due date: Sunday, May 11th, 2025 by 11:59pm, submitted through Gradescope.

Names:

Frank no partner

In this lab we will start exploring how photometry can teach us about the fundamental properties of stars. To do this we'll start by considering their spectra, and what we can learn about the spectra from two images in different filters.

Part 1: Stellar Spectra

The first part of this lab will be an introduction to spectra. Astronomical data typically comes in two basic forms: images and spectra. You have already seen images. Spectra are a bit different: we lose at least some of our ability to resolve an object and see its shape, but we spread out its light into its wavelengths. The intro slides to this lab included a spectrum as it appears on a detector. It is a colored spectrum of the Sun, showing many black parts where there is less light. These are absorption lines and give a whole lot of information about the physics of the Sun.

Just like Source Extractor takes the images and extracts the positions and brightnesses of stars, we can extract the flux of a star as a function of wavelength.

For this part of the lab, you will use three spectra contained in the files

`spectrum1.dat`, `spectrum2.dat`, and `spectrum3.dat`. The spectra are of three stars. The data files have two columns: the first is wavelength in Angstroms ($1 \text{ \AA} = 10^{-10} \text{ m}$), and the second is flux density in units of $\text{erg/s/cm}^2/\text{\AA}$.

You may read the files using, e.g.,

```
spectrum = np.loadtxt(specfile)
lam = spectrum[:, 0]
flam = spectrum[:, 1]
```

Please be aware that `lambda` has a special meaning within python, so don't try to define it as a variable!

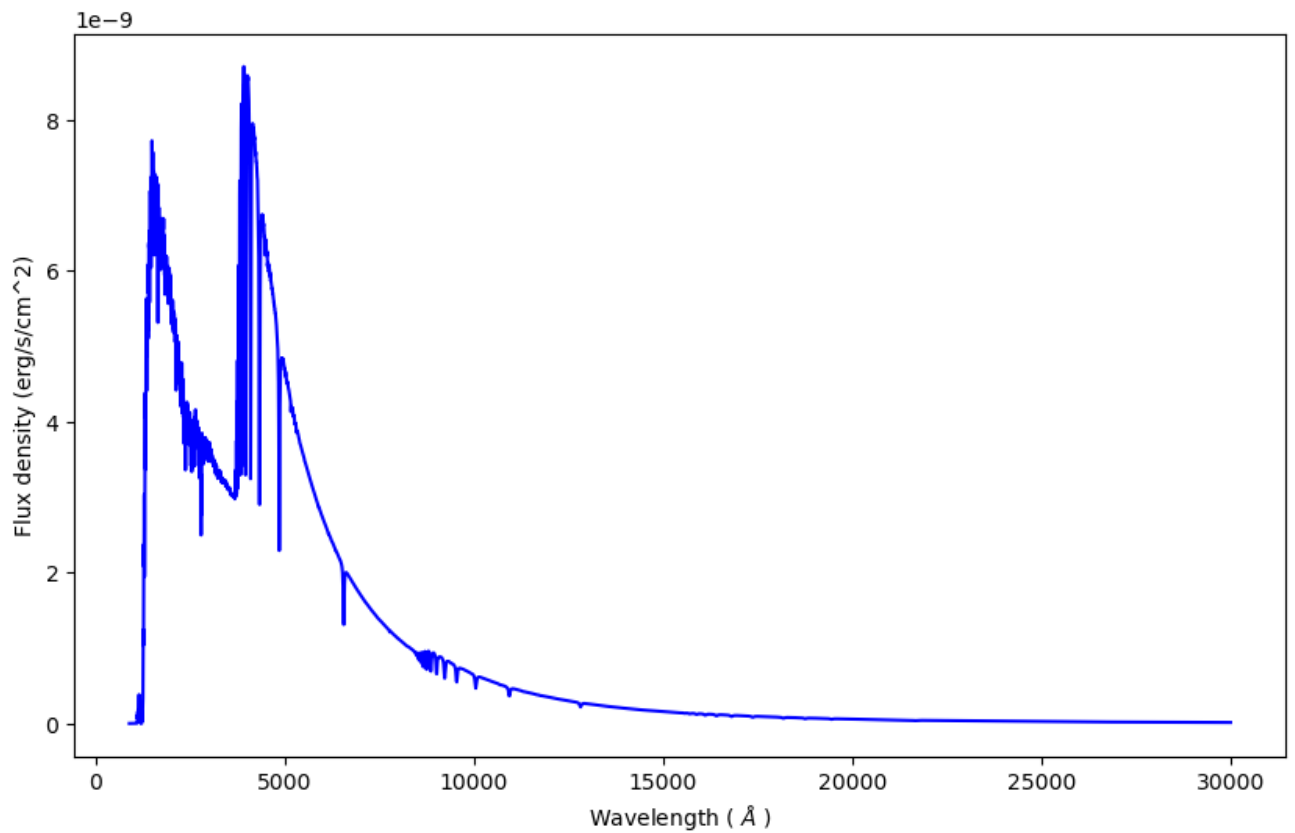
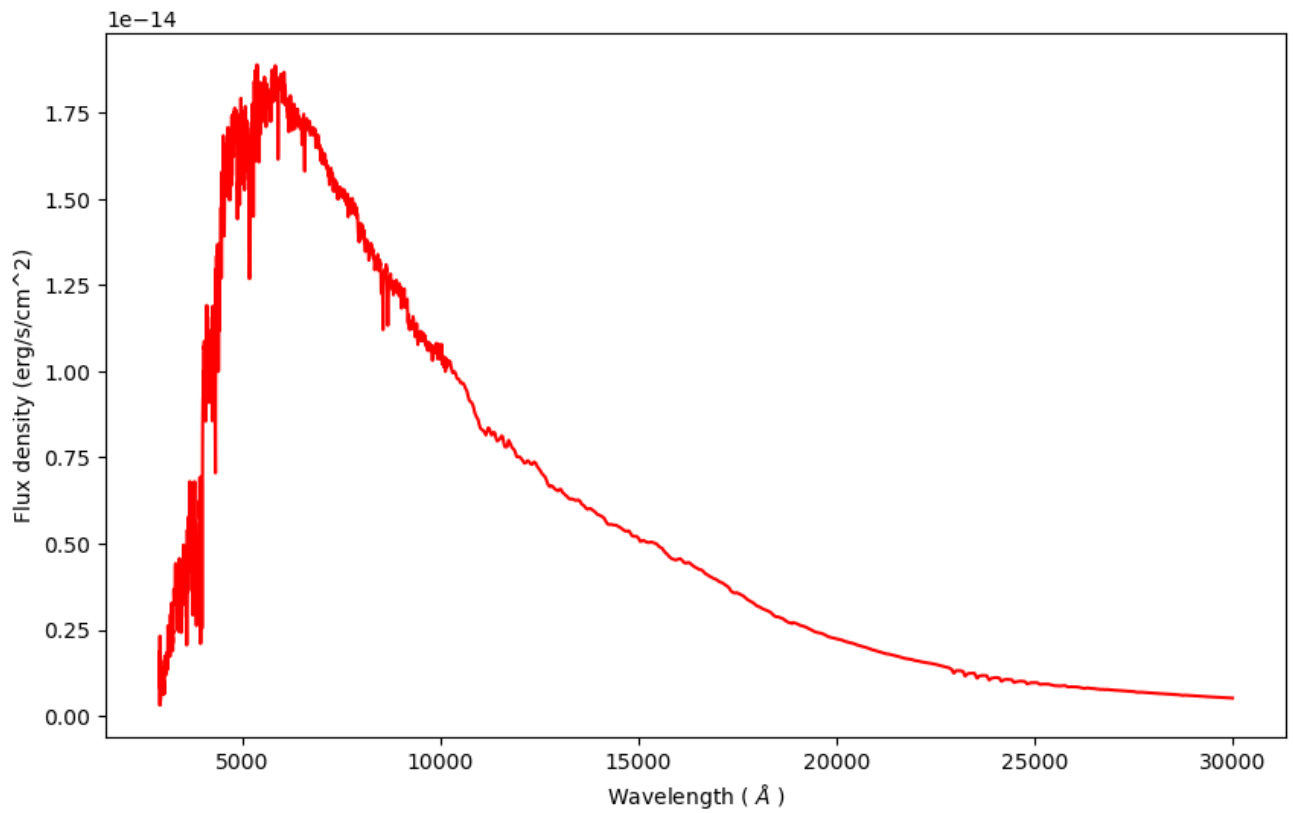
Make a plot of each star's spectrum as a function of wavelength. Please label the figures nicely using appropriate labels. For example:

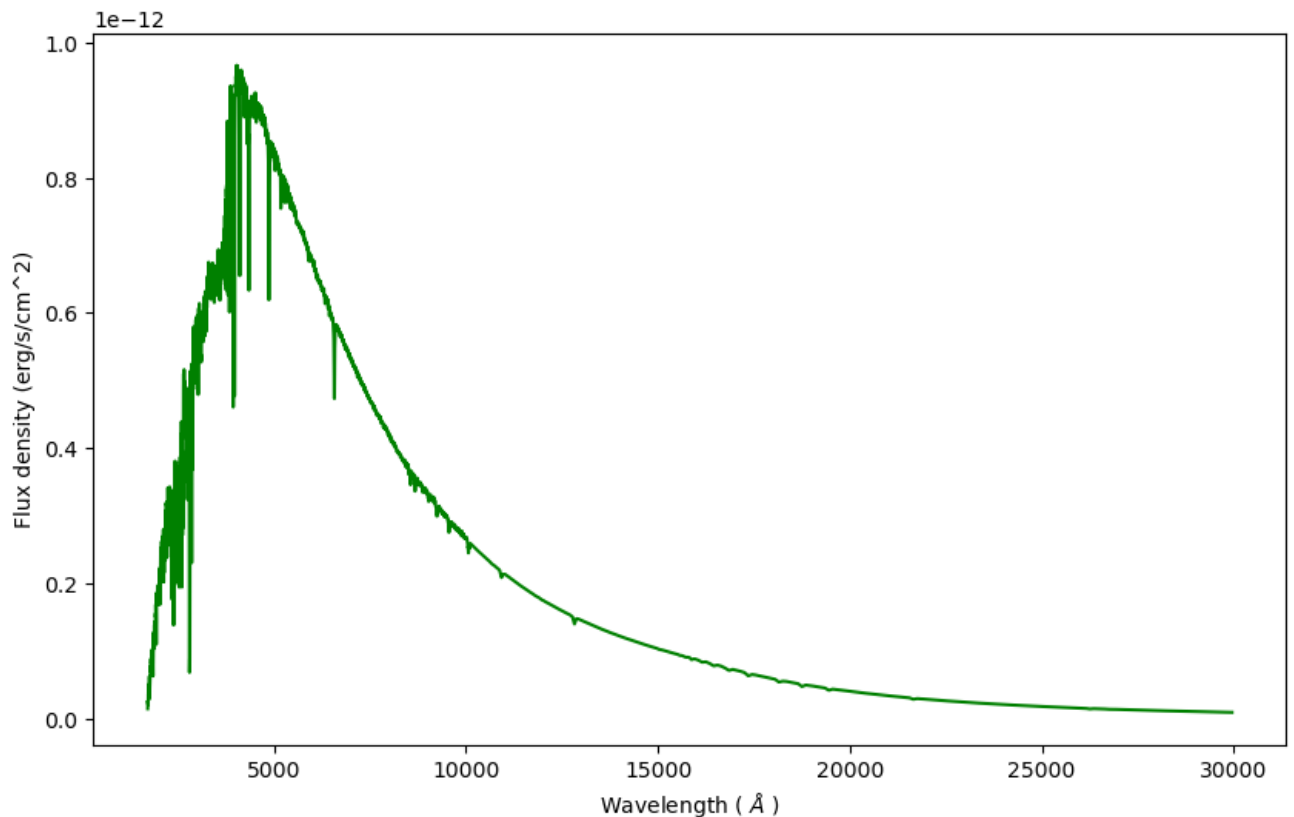
```
plt.xlabel(r'Wavelength (  $\text{\AA}$  )')
```

The `r` before the string tells matplotlib that it should use Latex to render this. Please get the units right for the vertical axis. Hint: if you are unsure of how to write a symbol in Latex, try <http://detexify.kirelabs.org>.

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
spec1 = np.loadtxt('spectrum1.dat')
spec2 = np.loadtxt('spectrum2.dat')
spec3 = np.loadtxt('spectrum3.dat')
specs = (spec1, spec2, spec3)
lam1 = spec1[:, 0]
flam1 = spec1[:, 1]
lam2 = spec2[:, 0]
flam2 = spec2[:, 1]
lam3 = spec3[:, 0]
flam3 = spec3[:, 1]
plt.figure(figsize=(10,6))
plt.plot(lam1, flam1, 'r')
plt.xlabel(r'Wavelength (  $\text{\AA}$  )')
plt.ylabel(r'Flux density (erg/s/cm^2)')
plt.show
plt.figure(figsize=(10,6))
plt.plot(lam2, flam2, 'b')
plt.xlabel(r'Wavelength (  $\text{\AA}$  )')
plt.ylabel(r'Flux density (erg/s/cm^2)')
plt.show
plt.figure(figsize=(10,6))
plt.plot(lam3, flam3, 'g')
plt.xlabel(r'Wavelength (  $\text{\AA}$  )')
plt.ylabel(r'Flux density (erg/s/cm^2)')
plt.show
```

```
Out[1]: <function matplotlib.pyplot.show(close=None, block=None)>
```





Now let's try to learn something about these stars. We can try to approximate the stars as blackbodies using the Planck function (look it up in Wikipedia). **Write down the Planck function in a markdown cell in your notebook so that it has as close as possible to the same units as the flux density of your spectra (i.e.~make sure it's per Angstrom, not per Hertz or per nm).** The units won't be completely identical: the Planck function will still be per steradian. This will be important soon.

$$B(\lambda, T) = \frac{2hc^2}{(\lambda)^5 * (\exp(hc/(\lambda kT) - 1))} \text{ units of erg/s/cm}^2/\text{angstrom/ster}$$

By eye, find the temperature and normalization of the Planck function that best fits each stellar spectrum. You should choose the temperature that gives a good fit to the peak wavelength (a blackbody isn't a great fit to `spectrum2.dat` for reasons that we will discuss later), and multiply the Planck function by a number to put it on the same scale as your spectrum. This number will be very small. Make a plot with the three stellar spectra again, with your best-fit Planck functions overplotted.

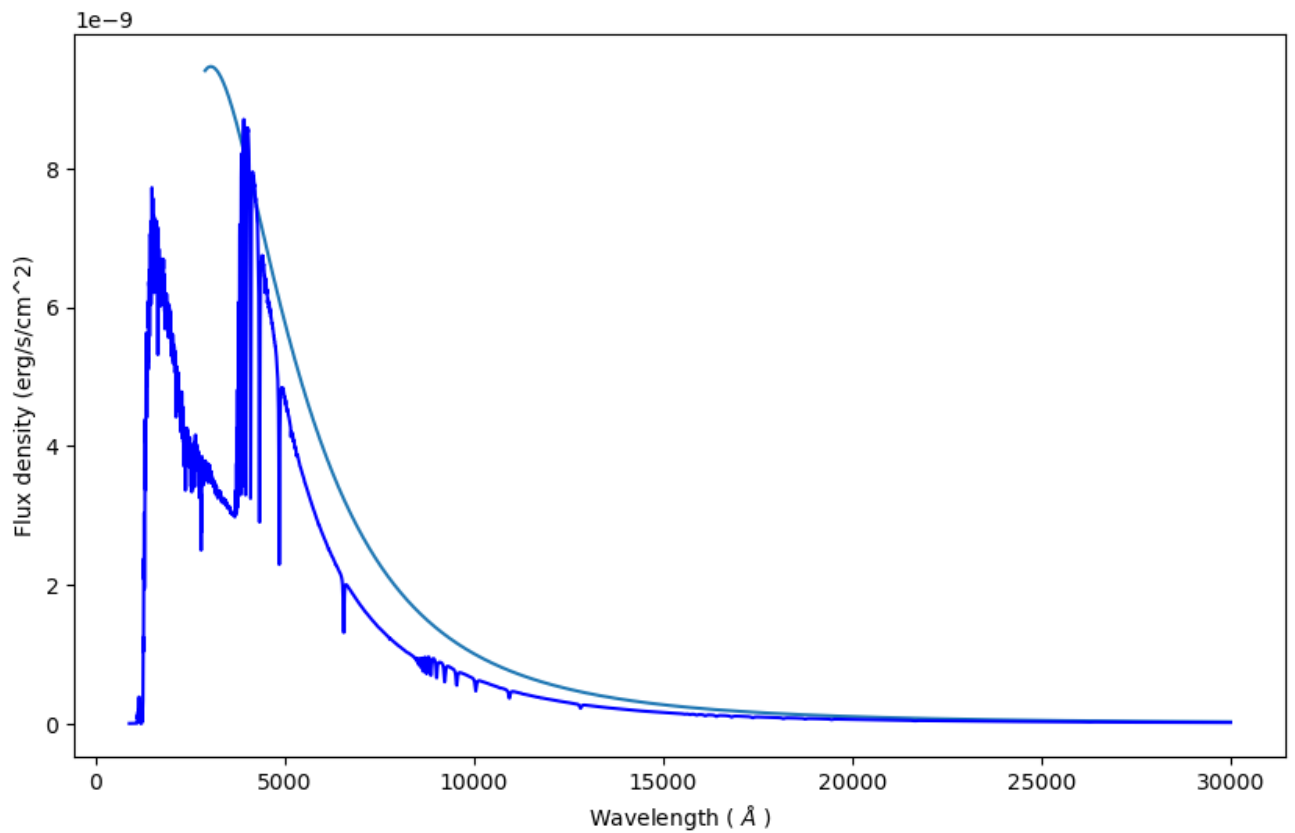
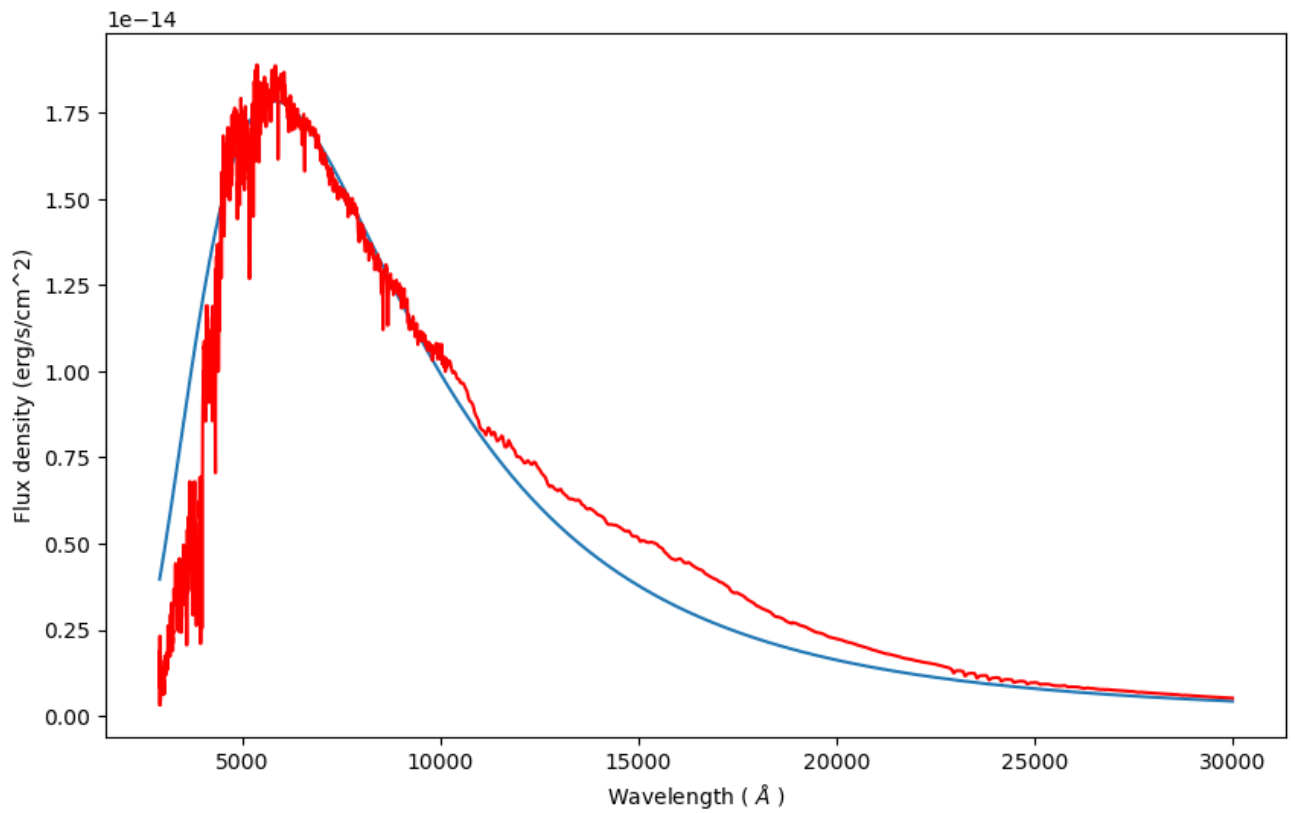
```
In [2]: import numpy as np
import matplotlib.pyplot as plt
spec1 = np.loadtxt('spectrum1.dat')
spec2 = np.loadtxt('spectrum2.dat')
spec3 = np.loadtxt('spectrum3.dat')
```

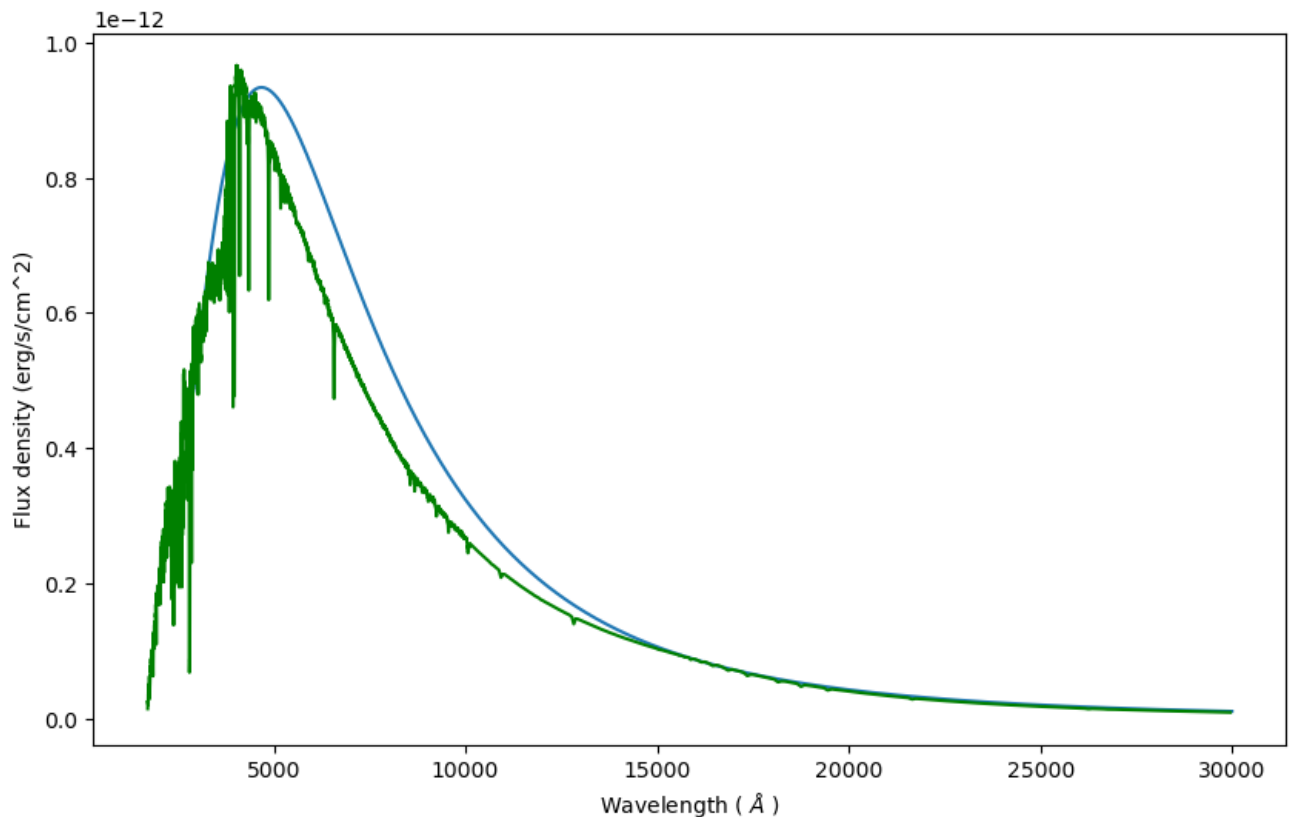
```

specs = (spec1, spec2, spec3)
lam1 = spec1[:, 0]
flam1 = spec1[:, 1]
lam2 = spec2[:, 0]
flam2 = spec2[:, 1]
lam3 = spec3[:, 0]
flam3 = spec3[:, 1]
def planckfunc(norm, lam, T):
    h = 6.626 * 10 ** -27 #erg*s
    k = 1.38 * 10 ** -16 #erg/k
    c = 3e10
    lam_u = lam * 10 ** -8 #fixing lam units to cm
    b = lam_u*k*T
    nm = 2 * h * (c ** 2)
    dn = (lam_u ** 5) * (np.exp(h*c/(b)) - 1)
    return (norm * nm)/dn
plt.figure(figsize=(10,6))
plt.plot(lam1, planckfunc(1.4e-28, lam1, 5000))
plt.plot(lam1, flam1, 'r')
plt.xlabel(r'Wavelength ( $ \AA $ )')
plt.ylabel(r'Flux density (erg/s/cm^2)')
plt.show
plt.figure(figsize=(10,6))
plt.plot(lam1, planckfunc(3e-24, lam1, 9500))
plt.plot(lam2, flam2, 'b')
plt.xlabel(r'Wavelength ( $ \AA $ )')
plt.ylabel(r'Flux density (erg/s/cm^2)')
plt.show
plt.figure(figsize=(10,6))
plt.plot(lam1, planckfunc(2.5e-27, lam1, 6200))
plt.plot(lam3, flam3, 'g')
plt.xlabel(r'Wavelength ( $ \AA $ )')
plt.ylabel(r'Flux density (erg/s/cm^2)')
plt.show

```

Out[2]: <function matplotlib.pyplot.show(close=None, block=None)>





You have fit two parameters by eye to each star's spectrum: the temperature and a normalization factor to the Planck function. What units does your normalization constant need to have in order for the units to work out? What is the physical meaning of this value?

The normalization constant will be in units of steradians, this normalization corresponds to how big the object looks from our perspective on earth.

Part 2: Stellar Colors

Many factors influence the appearance of a star's emitted energy spectrum, but the most important is the stellar temperature. You already saw this in Part 1: the three stars have almost the same chemical composition, but their spectra look very different. Their surface temperatures range from around 4000K to around 10000K: Spectrum 1 is from a ~ 5000 K star, Spectrum 2 is from a ~ 9500 K star, and Spectrum 3 is from a ~ 6200 K star.

The cluster images that you have been looking at, however, are not spectra. The telescope takes the light from the stars, focuses it, and uses a filter to let only some

colors through. Only photons within a specified wavelength range actually make it onto the detector. Here we will investigate how the visible-light spectrum changes with temperature, and how these changes are manifested in the stars' photometric colors.

The Lab 5 directory contains three filter files and three additional stellar spectra. The stars are at three different temperatures:

`f05.dat` = 9550K

`f10.dat` = 6110K

`f15.dat` = 4350K

These new files contain the energy flux F_λ (proportional to $\text{erg s}^{-1} \text{cm}^{-2} \text{\AA}^{-1}$), where wavelength λ is measured in Angstroms ($\text{\AA}$), and 1 Angstrom = 0.1 nm. The flux values have been arbitrarily normalized so they have a value of 100 units near 5500, $\text{\AA}$. Remember from Part 1 that the normalization of the stellar spectrum told you the angular diameter of the star on the sky. The shape of the spectrum tells you about the physical conditions of the star, mainly its temperature and chemical composition.

Take the spectra of the three stars `f05.dat`, `f10.dat`, and `f15.dat`, and plot them together on the same graph, using different colors. Also plot the Planck functions at these three temperatures on the same axes. You will notice that the hottest star is a particularly poor match to the blackbody; this is due to the hydrogen in the star's atmosphere.

```
In [3]: import numpy as np
import matplotlib.pyplot as plt
h = 6.626e-27
k = 1.381e-16
c = 3e10
def planckfunc(lam, T):
    h = 6.626 * 10 ** -27 #erg*s
    k = 1.38 * 10 ** -16 #erg/k
    c = 3e10
    lam_u = lam * 10 ** -8 #fixing lam units to cm
    b = lam_u*k*T
    nm = 2 * h * (c ** 2)
    dn = (lam_u ** 5) * (np.exp(h*c/(b)) - 1)
    return (nm)/dn * 1e-8 #cm to angstrom
#ai helped organize by color and fix stars data type
stars = {'f05.dat': 9550, 'f10.dat': 6110, 'f15.dat': 4350}
colors = ['tab:red', 'tab:green', 'tab:blue']
lam0 = 5500
plt.figure(figsize=(10,6))
```

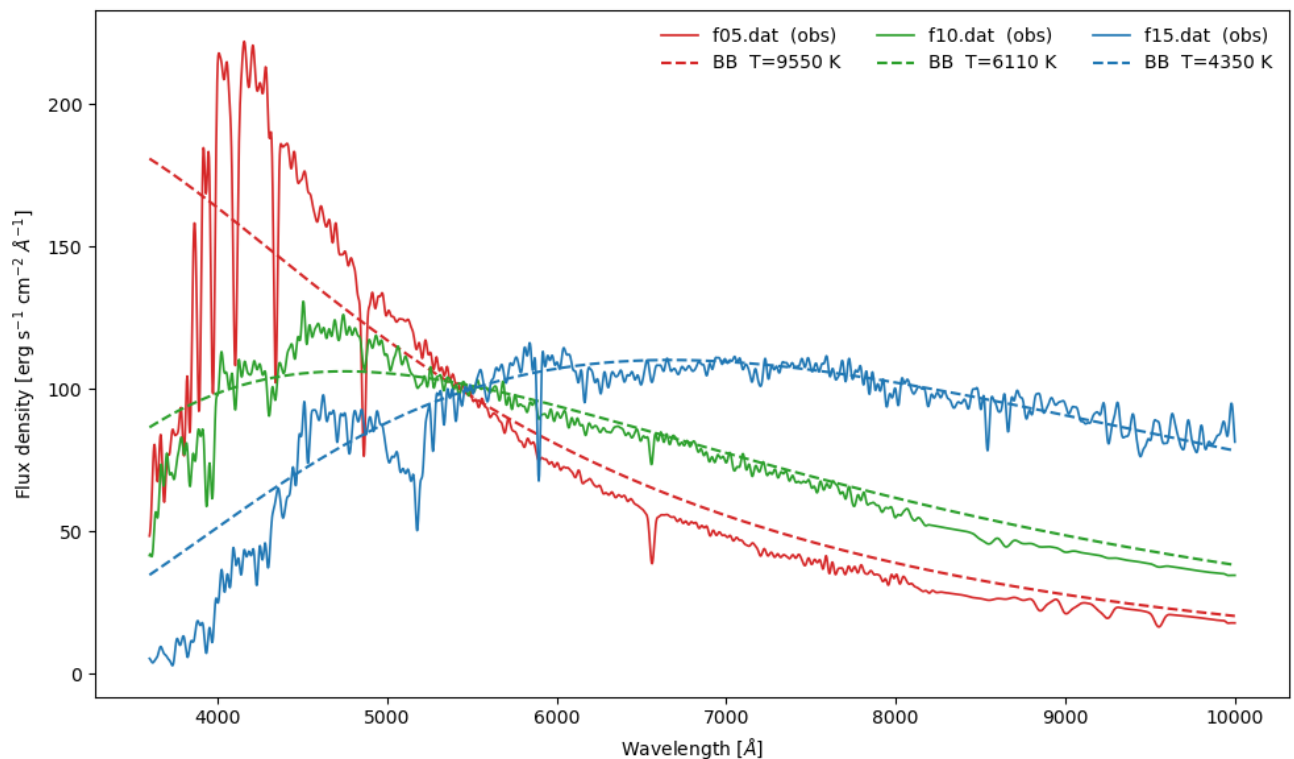


```

for (fname, T), col in zip(stars.items(), colors): #ai helped create for loc
    #with scaling
    data = np.loadtxt(fname)
    lam = data[:,0]
    F = data[:,1]
    B = planckfunc(lam, T)
    idx = np.abs(lam - lam0).argmin()
    scale = F[idx] / B[idx]
    B_scaled = B * scale
    plt.plot(lam, F, color=col, lw=1.2, label=f'{fname} (obs)')
    plt.plot(lam, B_scaled, color=col, ls='--', label=f'BB T={T} K')

plt.xlabel(r'Wavelength [$\AA$]') #ai created plotlabels
plt.ylabel(r'Flux density [erg s$^{-1}$ cm$^{-2}$ $\AA^{-1}$]')
plt.legend(frameon=False, ncol=3)
plt.tight_layout()
plt.show()
#as seen below, the lowest temp matches up best

```



Using a CCD detector with colored filters, we measure the energy flux from stars integrated over the bandpass of each filter:

$$F = \int_0^{\infty} T[\lambda] F_{\lambda} d\lambda,$$

where $T[\lambda]$ is called the *transfer function* and describes how efficiently we record energy from photons of a given wavelength. $T[\lambda] = 1$ means that our instrument

records every photon at a given wavelength, while $T[\lambda] = 0$ means that it never records a single photon. **We will do the integrals for the filters B and V .** It's worth spending a bit of time on how to do an integral numerically, as this is something many of you will have to do again and again.

The integral is defined as the limit of the Riemann sum,

$$\int_a^b f(x)dx = \lim_{N \rightarrow \infty} \sum_{i=0}^{N-1} f[x_i] \Delta x_i,$$

where Δx_i is the spacing between values of x_i , $x_0 = a$, and $x_N = b$. Let's see how we can write this in python. First, we'll define x to run from a to b with $N + 1$ points in total. For now, define $a = 0$, $b = 1$, and $N = 1000$ (we'll change these later when we actually integrate over the band passes).

```
x = np.linspace(a, b, N + 1)
```

Now check: use the print function to verify that `x[0] = a` and `x[N] = b`. In your case, the spacing is uniform, so Δx_i is the same for all points i .

```
In [4]: a=0
b=1
N=1000
x = np.linspace(a, b, N + 1)
print(x[0], x[N])
```

```
0.0 1.0
```

Now use your value for Δx , the `numpy` function `sum`, and your values of `x` to numerically evaluate the integral

$$\int_0^1 \sin x dx$$

and compare to the actual value of the integral. Remember that your (left) Riemann sum should stop at $N - 1$, which you can write using index notation as `[:N]`. Show that your answer is closer to the truth if $N = 2000$, but is farther from the truth if $N = 500$.

Now we can do an integral numerically, but how do we integrate

$$F = \int_0^\infty T[\lambda] F_\lambda d\lambda?$$

We need to define a wavelength array, and then be able to call $T[\lambda]$ and $F[\lambda]$ just as we

did above with sine. We can do this by first reading in the data files and then *interpolating* between the data points. We will use the package `scipy.interpolate`, and we will use cubic polynomials (called *splines*) to interpolate:

```
from scipy import interpolate
```

From this point, `python` actually makes our lives pretty easy. First, let's read in one of the spectral files (first column is wavelength, second column is flux density F_λ):

```
spec_data = np.loadtxt('f05.dat')
```

and then

```
Flam = interpolate.interp1d(spec_data[:, 0], spec_data[:, 1],
                             kind='cubic')
```

Now `Flam` is a function that you can call just like `np.sin`. Calling this function will return an error if you ask it for the value outside the range of x -values you supplied to `interpolate.interp1d`. **For the transfer function, we may assume that it is zero outside of the wavelength range given. Read the documentation page (check the keywords `bounds_error` and `fill_value`) to make the function behave in this way.**

Now we are ready to compute magnitudes. **Use the tools you have developed above to integrate the three spectra over the transfer functions given by `Bfilt.dat` and `Vfilt.dat`.** The definition of magnitude includes a zero point:

$$M = -2.5 \log_{10} F - Z.$$

For now, ignore the zero points Z (let $Z = 0$). What are the magnitudes of the three stars in the two bands? Please fill in the following table. Also, please compute the difference between the B and V band magnitudes (the $B - V$ color). This is the ratio between the flux in the B band and the flux in the V band.

```
In [5]: import numpy as np
        from scipy import interpolate
        def band_flux(lam_star, F_lam, lam_filt, T_filt): #ai helped create bounds for
            T_interp = np.interp(lam_star, lam_filt, T_filt, left=0.0, right=0.0)
            return np.trapz(T_interp * F_lam, lam_star)

        spectra = {}
        for fname in stars: #ai helped create loops for proper indexing
            lam, F = np.loadtxt(fname, unpack=True)
```

```

spectra[fname] = (lam, F)
stars = {'f05.dat': 9550, 'f10.dat': 6110, 'f15.dat': 4350}
lam_B, T_B = np.loadtxt('Bfilt.dat', unpack=True)
lam_V, T_V = np.loadtxt('Vfilt.dat', unpack=True)
results = {}
for fname, (lam, F) in spectra.items():# ai used to create for loops and tab
    F_B = band_flux(lam, F, lam_B, T_B)
    F_V = band_flux(lam, F, lam_V, T_V)
    results[fname] = {'F_B': F_B, 'F_V': F_V}

for r in results.values():
    r['M_B'] = -2.5 * np.log10(r['F_B'])
    r['M_V'] = -2.5 * np.log10(r['F_V'])
    r['B-V'] = r['M_B'] - r['M_V']

print("\nJohnson magnitudes (zero-point Z = 0):")#ai used for loops and tabl
print("{:8s} {:>12s} {:>12s} {:>8s}".format("star", "M_B", "M_V", "B-V"))
print("-" * 42)
for fname in stars:
    r = results[fname]
    print(f"{fname:8s} {r['M_B']:12.3f} {r['M_V']:12.3f} {r['B-V']:8.3f}")

```

Johnson magnitudes (zero-point Z = 0):

star	M _B	M _V	B-V
f05.dat	-13.000	-12.361	-0.640
f10.dat	-12.523	-12.377	-0.146
f15.dat	-11.867	-12.313	0.446

---	f05.dat	f10.dat	f15.dat1
B	-13	-12.361	-.64
V	-12.523	-12.377	-.146
B-V	-11.867	-12.313	.446

Does a high value of $B - V$ indicate a star that is very blue (puts out lots of blue light relative to yellow/green light), or a star that is very red?

High B-V means the V band is brighter so the star will emit more yellow/green light and be more red, where a smaller B-V corresponds to a bluer star

What is the qualitative relation between temperature and B - V color?

At temperature increases, B-V will decrease corresponding to more blue flux or blue light. Cooler stars will have a larger B band or a larger B-V value.

Now you can imagine that only using images of a star in two filters, we can learn a lot about its fundamental properties!

So colors provide a way to estimate stellar temperatures. Suppose we know the temperature -- then what else do we know? Stellar evolution theory provides relations between the characteristics of stars at the time they are born (mass and composition), and their properties (including temperature and luminosity) at any time in their later lives. We'll explore this more in Lab 6. For a quick introduction to (or refresher on) many of the basic notions, see [here](#).